

The Tally Sheet Model

How a machine can learn to turn one list of bits into another — using nothing but counters that go up by one. No calculus, no neural networks, no jargon assumed.

WHAT'S IN HERE

1. The job we're trying to do
2. Our tiny worked example
3. Two obvious approaches, and why both fail
4. The idea: situations
5. What a slot actually is (tally sheets)
6. Learning, if life were easy: pure counting
7. Reading the model: how it answers a question
8. Why the useless bits get ignored automatically
9. The real problem: nobody labels the data
10. EM: counting when you don't know who's who
11. Is this a Bayesian network? Is this Bayesian inference?
12. Why this doesn't explode
13. Where it goes wrong, and split-merge
14. Glossary

1. The job we're trying to do

You have pairs of things. Each pair is an **input** and an **output**, and both of them are just lists of on/off switches — bits.

You want a machine that, when you show it a new input, tells you what the output should be.

Two things make this harder than a lookup table:

1. **Some input bits don't matter.** Flip them and the answer stays the same. You don't know which ones in advance — you want the machine to work that out.
2. **One input can have several correct outputs.** Not one answer — a small menu of valid answers. You want all of them, listed separately, not mashed together.

Everything in this document is in service of those two requirements.

2. Our tiny worked example

Real versions of this have 100 bits on each side. That's impossible to follow by hand, so here is a miniature with **4 input bits** (call them A, B, C, D) and **4 output bits** (W, X, Y, Z), and only 8 examples.

This is our entire training set:

EXAMPLE	A	B	C	D	W	X	Y	Z
1	1	0	0	1	1	1	0	0
2	1	1	0	0	1	1	0	0
3	1	1	0	1	0	0	1	0
4	1	0	0	0	0	0	1	0
5	0	1	1	0	0	0	0	1
6	0	0	1	1	0	0	0	1
7	0	0	1	0	0	0	0	1
8	0	1	1	1	0	0	0	1

Eight training examples. This is all the machine ever sees — no labels, no explanations, no hints.

Stare at it for a moment and you might spot the pattern. Here it is written out — but remember, **the machine is never told any of this**. This is the answer key, for your benefit only:

THE HIDDEN TRUTH

When A is on → the output is either W, X or Y. Both are correct. Which one you get is a coin flip.

When C is on → the output is always Z.

B and D are pure noise. They're on about half the time and mean absolutely nothing.

So this tiny dataset contains both of our hard requirements: two of the four input bits are useless, and one input pattern legitimately has two different correct answers.

3. Two obvious approaches, and why both fail

Approach one: memorize every input

Build a giant table. One row for every possible input, and in that row write down the output.

With 4 input bits there are 16 possible inputs — fine. With 100 input bits there are 2^{100} possible inputs. That number is about 1,000,000,000,000,000,000,000,000,000. You would need more rows than there are atoms in your body, and you'd have to see each input at least once to fill in its row.

Dead on arrival. This is the "explosion" — and avoiding it is the reason everything else in this document exists.

Approach two: work out each output bit separately

Forget whole patterns. Just learn, for each output bit, how often it's on.

Take our data. When A is on (examples 1–4), how often is each output bit on?

```
W: on in 2 of 4 → 50%
X: on in 2 of 4 → 50%
Y: on in 2 of 4 → 50%
Z: on in 0 of 4 → 0%
```

Every one of those numbers is **correct**. And the machine is now useless.

Because if you ask it "so what's the output?" it says "W is 50% likely, X is 50% likely, Y is 50% likely." Pick the most likely bits and you get W, X, Y — which is **not a valid answer**. It never appeared in the data. It's a blend of two real answers, and the blend is wrong.

THE CORE FAILURE

Three separate percentages cannot express "W and X arrive together as a package, and Y arrives alone."

The information about which bits **travel together** was thrown away the moment we started tracking bits one at a time. Averaging destroys exactly the thing we care about.

Think of a restaurant that only sells fixed combo meals. If I tell you "50% of our combos contain fries, 50% contain a milkshake," those are true statistics — but you can't reconstruct the menu from them, and if you pick items by flipping a coin for each, you'll order a combo that doesn't exist.

4. The idea: situations

Here's the way out.

Instead of memorizing every possible input, or tracking every bit separately, we assume something about the world:

THE ASSUMPTION EVERYTHING RESTS ON

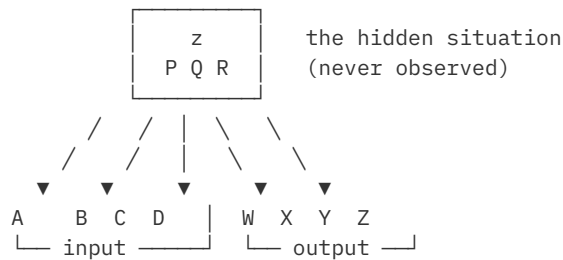
There are a **small number of underlying situations**. Each situation produces a characteristic-looking input and one specific output pattern. Every example in your data came from one of them.

In our tiny dataset there are three:

- Situation P: input has A on, output is **W,X** (examples 1, 2)
- Situation Q: input has A on, output is **Y** (examples 3, 4)
- Situation R: input has C on, output is **Z** (examples 5, 6, 7, 8)

Notice that P and Q **look identical on the input side** — both have A on — but produce different outputs. That's not a bug. That's how the machine will represent "this input has two valid answers."

The situation is called a **hidden variable**, usually written **z**. "Hidden" means exactly what it sounds like: it's not written anywhere in your data. Nobody stamped example 3 with a label saying "I'm situation Q." The machine has to figure out that situations exist at all, and which example belongs to which.



Read the arrows as "causes" or "produces". The situation is upstream of everything you can see. Both the input bits and the output bits are consequences of it.

This single picture is what solves the "bits travel together" problem. W and X go together because they share a cause — when the situation is P, it switches both of them on.

5. What a slot actually is (tally sheets)

Now the concrete part. A slot is the machine's storage for one situation. It is nothing more exotic than a page of counters.

Each slot has:

- One counter for **how many examples landed in this slot**.
- One counter **per bit**, for how many of those examples had that bit switched on.

That's the whole thing. Nine counters per slot in our example (one total + eight bits). A slot is a tally sheet.

Here's what the three slots look like once filled in. Read 2/2 as "2 out of the 2 examples in this slot."

SLOT	TOTAL	A	B	C	D	W	X	Y	Z
P	2	2/2	1/2	0/2	1/2	2/2	2/2	0/2	0/2
Q	2	2/2	1/2	0/2	1/2	0/2	0/2	2/2	0/2
R	4	0/4	2/4	4/4	2/4	0/4	0/4	0/4	4/4

Three tally sheets. Green columns are bits that matter; amber columns are the noise bits. Look at what's different about them.

Two things are worth pausing on, because they're the entire point of the design.

The noise bits are visibly flat

Look down the B column: 1/2, 1/2, 2/4. That's 50%, 50%, 50%. Identical in every slot.

Now look down the A column: 100%, 100%, 0%. Wildly different between slots.

REQUIREMENT 1, SOLVED

A bit that **varies across slots** is telling you something. A bit that is **the same in every slot** is telling you nothing.

You never had to instruct the machine which bits matter. Fill in the tallies and it's sitting there in the table, readable by eye. Sort the columns by how much they vary and you have your relevance ranking.

P and Q are twins on the input side

Slots P and Q have exactly the same input tallies. They only differ on the output side. So when an input with A on arrives, both slots match it equally well — and the machine will offer **both** of their outputs as options.

REQUIREMENT 2, SOLVED

A menu of several valid answers is stored as **several slots that agree about the input and disagree about the output**.

IMPORTANT VOCABULARY

A slot is **one situation-and-output pair**, not one situation. "When A is on" needed two slots (P and Q) because it has two possible outputs. Each slot can only hold one output pattern.

You'll also see slots called **states**, **components**, or "values of **z**". Same thing.

6. Learning, if life were easy: pure counting

Suppose for a moment that every example came with a label saying which slot it belongs to. Then training the machine is this, and only this:

- 1 Set every counter on every tally sheet to zero.
- 2 For each training example: look at its label, find that slot's tally sheet, add 1 to its total, and add 1 to the counter of every bit that's switched on.
- 3 There is no step 3. You're finished.

Watch it happen for slot P, which gets examples 1 and 2:

start	total=0	A=0	B=0	C=0	D=0		W=0	X=0	Y=0	Z=0
example 1	1 0 0 1		1 1 0 0	← A and D on; W and X on						
after	total=1	A=1	B=0	C=0	D=1		W=1	X=1	Y=0	Z=0
example 2	1 1 0 0		1 1 0 0	← A and B on; W and X on						
after	total=2	A=2	B=1	C=0	D=1		W=2	X=2	Y=0	Z=0

That's the finished tally sheet for slot P, matching the table above. **This is what "learning by counting" means.** No equations, no gradual adjustment, no training loop. Counters go up by one.

WHY WE KEEP TALLIES, NOT PERCENTAGES

We could store "A is on 100% of the time" instead of "A: 2 out of 2." The percentage is what we actually use when answering questions — but the **tally is what we store**, for one big reason:

Tallies can be **added to**. A new example arrives tomorrow? Add 1 to the relevant counters. Done — and every other slot's counters are untouched, exactly, provably. If you'd only kept "100%", you'd have no idea how much weight that percentage carried and no correct way to fold new data into it.

This is what lets the machine learn a new thing without disturbing what it already knows.

One wrinkle: never let a counter sit at zero

Look at slot P's counter for C: it's $0/2$. Taken literally that says "in situation P, bit C is on 0% of the time — impossible, never, under any circumstances."

That's too strong a claim from two examples, and it causes a nasty failure: anything with C switched on gets a score of exactly zero for slot P, forever, and can never be reconsidered.

The fix is one line: **start every counter at 1 instead of 0**. Pretend you saw each bit on once and off once before you began. Now "never" becomes "very unlikely," which is honest and can be revised by evidence.

This is called smoothing or pseudo-counts. It's also, quietly, the one genuinely Bayesian ingredient in the whole recipe — more on that in section 11.

7. Reading the model: how it answers a question

The machine is trained. Now someone hands it a new input and wants an answer.

New input: A=1, B=1, C=0, D=0.

The question the machine asks itself is: "which situation was I probably in?" It scores every slot by asking how well that slot's tally sheet predicts what it's seeing.

The rule for scoring is: **go through the input bits one at a time and multiply**. For a bit that's ON, multiply by how often that slot has it on. For a bit that's OFF, multiply by how often that slot has it off.

Slot P's input tallies are A: 2/2, B: 1/2, C: 0/2, D: 1/2 — that's 100%, 50%, 0%, 50%.

```
slot P
A is on   → × 1.00   (P has A on 100% of the time)
B is on   → × 0.50
C is off  → × 1.00   (P has C off 100% of the time)
D is off  → × 0.50

                = 0.25
```

```
slot Q — identical input tallies to P
1.00 × 0.50 × 1.00 × 0.50 = 0.25
```

```
slot R — has A on 0% of the time, so:
0.00 × 0.50 × 0.00 × 0.50 = 0.00
```

Then weight each score by how common that slot is overall (P and Q hold 2 of 8 examples each, R holds 4 of 8), and scale the three numbers so they add up to 100%:

```
slot P: 0.25 × 2/8 = 0.0625 → 50%
slot Q: 0.25 × 2/8 = 0.0625 → 50%
slot R: 0.00 × 4/8 = 0.0000 → 0%
```

Now read off the answer. Slot P says the output is **W,X**. Slot Q says the output is **Y**. Slot R isn't in the running.

THE MACHINE'S ANSWER

"Two possibilities. Either **W,X** — 50% likely. Or **Y** — 50% likely."

That's exactly right, and it's exactly what we wanted: a menu, with weights, not a blend.
Compare it to the mush from section 3 that suggested **W,X,Y** — an answer that doesn't exist.

8. Why the useless bits get ignored automatically

This deserves its own section because it's the most elegant part, and it happens without anyone arranging it.

Go back to the scoring. Bit B was switched on in our query. Look at what B contributed:


```
slot P:   × 0.50   ← B's contribution
slot Q:   × 0.50   ← the same
slot R:   × 0.50   ← the same
```

B multiplied **every slot by the same number**. And in the final step we scaled everything so the scores add to 100% — which cancels out any factor that's common to all of them.

B had literally zero effect on the answer. Not a small effect. Zero. The same is true of D.

REQUIREMENT 1, MECHANICALLY

A bit whose tallies are flat across all slots multiplies every slot equally, and equal factors vanish when you normalize. Useless bits cancel themselves out.

Nobody wrote a rule to ignore B and D. Nobody tuned a knob. It falls out of counting honestly and then scoring by multiplication.

Meanwhile bit A, whose tallies are 100%/100%/0%, multiplied slot R by zero and the others by one. It did all the work of deciding the answer. The bits that carry information are precisely the bits that distinguish slots — which is why "how much does this bit vary across slots" is a genuine relevance score.

9. The real problem: nobody labels the data

Everything in section 6 assumed each example came with a tag saying which slot it belongs to.

It doesn't. You have eight rows of bits. That's all. No tags, no situation names, not even the knowledge that there are three situations.

And you're stuck in a loop:

If you knew which slot each example belongs to



you could just count. Easy. (section 6)

If you knew what the slots look like



you could work out which one each example belongs to. Easy. (section 7)

You know NEITHER.

Each one would hand you the other, and you have no way in.

This is a chicken-and-egg problem, and it is the only reason we need anything cleverer than counting.

10. EM: counting when you don't know who's who

EM stands for Expectation–Maximization. The name is unhelpful. Here's the entire idea:

THE WHOLE TRICK

Make up an answer to one side of the loop, use it to work out the other side, then go back and redo the first with the improved information. Repeat until nothing changes.

In the two steps:

- E

Guess the labels
Using the current tally sheets (however bad), score every example against every slot — exactly the scoring from section 7 — and record the result as a **fraction**, not a hard choice.
- M

Re-do the tallies
Wipe the counters and count again, using those fractions. An example that's 70% slot P adds **0.7** to slot P's counters and **0.3** to slot Q's.

Then repeat. That's it. Both steps are things we've already built: E is section 7, M is section 6.

Why fractions?

Because forcing each example into exactly one slot makes the machine brittle — an ambiguous example gets shoved into a bucket it may not belong in, and that mistake poisons the tallies. Fractional counting lets uncertain examples hedge, contributing a bit to each candidate, and lets the whole arrangement drift smoothly instead of lurching.

Yes, this means your counters hold numbers like 1.7 instead of 2. That's fine. They're still tallies; they're just tallies that accept partial credit.

Watching one round happen

Start with two slots and deliberately terrible made-up tally sheets. We'll write them as percentages for readability:

	A	B	C	D		W	X	Y	Z
slot 1	60%	50%	40%	50%		60%	50%	40%	50%
slot 2	40%	50%	60%	50%		40%	50%	50%	60%

These are nonsense — deliberately. Nobody looked at the data to produce them.

E step, for example 1, which is `A=1 B=0 C=0 D=1 | W=1 X=1 Y=0 Z=0`. Score it against both slots, using all eight bits this time (input and output, because during training we can see both):

slot 1:

A on	×0.60	B off	×0.50	C off	×0.60	D on	×0.50
W on	×0.60	X on	×0.50	Y off	×0.60	Z off	×0.50
							= 0.0081

slot 2:

A on	×0.40	B off	×0.50	C off	×0.40	D on	×0.50
W on	×0.40	X on	×0.50	Y off	×0.50	Z off	×0.40
							= 0.0016

scale so they add to 100%:

slot 1: 0.0081 / 0.0097 = 83.5%

slot 2: 0.0016 / 0.0097 = 16.5%

So example 1 is labelled "83.5% slot 1, 16.5% slot 2." Not because anything knows that's right — the tally sheets were nonsense — but because example 1 happens to lean slightly toward slot 1's random numbers.

M step. Do that for all eight examples, then count with the fractions:

example 1 is 83.5% slot 1, and has A switched on

- add 0.835 to slot 1's A counter
- add 0.835 to slot 1's total counter
- add 0.165 to slot 2's A counter
- add 0.165 to slot 2's total counter

... and the same for every bit of every example.

Now you have new tally sheets — slightly less random than before, because they were built from data instead of thin air. Go back to E and do it again with the improved sheets. The labels get slightly better. Then the sheets get slightly better. And so on.

Why starting from nonsense works

It seems like it shouldn't. But the random starting point is never perfectly balanced — some examples lean marginally toward slot 1, others toward slot 2. Counting sharpens that lean a little. The sharper sheets sort the examples a little more decisively. Which sharpens the sheets further. The whole thing bootstraps itself from an accidental wobble.

After a handful of rounds, our eight examples sort themselves cleanly and the tally sheets converge to the ones in section 5 — the true situations, discovered from data that never mentioned them.

THE GUARANTEE, AND ITS LIMIT

You can prove the model's fit to the data **never gets worse** from one round to the next. So it always settles. That's a real mathematical guarantee.

What it does not guarantee is that it settles in the best place. Picture walking uphill in thick fog: every step goes up, so you certainly reach a summit — possibly a small hill with the real mountain hidden next door. Where you end up depends on where you started.

A FREE BUG TEST

Print the fit score every round. It is guaranteed not to go down — so **if it ever goes down, you have a bug**. Almost no algorithm hands you a correctness check this cheap.

11. Is this a Bayesian network? Is this Bayesian inference?

Three separate questions that get tangled together. Let's untangle them.

Is this a Bayesian network?

Yes. A Bayesian network is just two things: a diagram showing what depends on what, and a table of numbers for each item saying how it behaves given whatever points at it.

Ours is the diagram from section 4 — **z** at the top, arrows down to every bit — and the tables are the tally sheets. That's a Bayesian network. A very simple shape of one, sometimes called a latent class model, but a genuine one.

Do we do Bayesian inference?

Yes — every time the machine answers a question.

Bayes' rule is the formal name for one commonsense move: you see some evidence, and you update your belief about the hidden cause behind it.

That's precisely section 7. We saw evidence (the input bits), we asked "which hidden situation would have produced this?", we scored each candidate, and we ended up with belief spread across them: 50% / 50% / 0%. That is Bayes' rule. You've been doing Bayesian inference since section 7 without the label.

So is this a "Bayesian method"?

Only a little, and this is the part that usually confuses people.

When people say "Bayesian machine learning" they usually mean something more demanding: keeping track of uncertainty about the model's own numbers. Not just "A is on 100% of the time in slot P" but "I believe it's 100%, give or take, and here's my uncertainty about that belief."

We don't do that. We commit to one set of tallies and use them. That's ordinary maximum-likelihood fitting, not full Bayesian learning.

The one exception is the smoothing from section 6 — starting every counter at 1 instead of 0. That is a Bayesian prior: a belief held before seeing any data ("nothing is truly impossible"), expressed as a nudge to the numbers. Which illustrates a useful general point:

DEMYSTIFYING PRIORS

A prior sounds heavy. In practice it's almost always just a **small adjustment added to your counts, or a penalty added to your objective**. "I believe nothing is impossible" becomes "start the counters at 1." "I believe most bits are irrelevant" becomes "charge a fee per bit used."

No integrals required. If someone tells you that you need heavy machinery to "be Bayesian" for a problem like this — you don't.

12. Why this doesn't explode

Back to the 2^{100} disaster from section 3. Why doesn't this design suffer it?

The rule is simple and worth memorizing:

THE RULE THAT GOVERNS COST

In a diagram like this, a node needs **one table row per combination of everything pointing at it**.

In the broken approach, each output bit had all 100 input bits pointing at it $\rightarrow 2^{100}$ rows. Catastrophe.

In our design, look at the arrows again. Every single observed bit — input and output alike — has **exactly one thing** pointing at it: **z**. So each bit needs one counter per slot. Not one per combination of other bits. One per slot.

storage needed:

number of slots	×	number of bits	
80	×	200	= 16,000 numbers

versus the table approach:

$2^{100} \approx 1,000,000,000,000,000,000,000,000,000$ rows

And crucially, growth is now **linear**. Double the input bits and you double the storage. Under the old scheme, one extra input bit doubled the whole table.

WHAT THE ESCAPE COSTS

The saving comes from an assumption: **once you know the situation, the bits don't influence each other**. Given slot P, bit A and bit B are unrelated.

That's a real restriction. Something like "bit 3 matters only when bit 7 is on" can't be said directly — you'd have to spend extra slots to cover the cases. So the explosion didn't vanish; it became a **dial** (the number of slots) that turns up as your data gets more tangled. For clustered, situation-shaped data it stays small. That's the bet.

13. Where it goes wrong, and split-merge

Remember EM's limit: it always walks uphill, but it might climb the wrong hill. Here's what the wrong hill actually looks like.

slot 12:		absorbed TWO situations, blurring them together
slot 37:	.	empty, never used

One slot doing double duty; another idle. The blurred slot outputs the average of two real answers — which, as we know from section 3, is not a real answer.

Why EM can't fix this itself

EM only accepts changes that improve the fit immediately. To pull the two situations apart it would have to move examples out of slot 12 one at a time — and moving the first one makes the fit **worse**. It only becomes better once you've moved about half of them.

There's a valley between where EM is and where it should be. EM walks uphill. It cannot cross valleys.

The fix: pick it up and move it

Split-merge is surgery on a settled model:

- **Split** — take an overloaded slot, cut its examples in half at random, hand half to an empty slot.
- **Merge** — take two slots doing the same job, fuse them, freeing one up.

Then run EM again from there. It doesn't walk across the valley; it **teleports to the other side** and lets EM climb from the new position.

The random half-split is deliberately crude — it has no idea which examples belong to which situation. It doesn't need to. It just needs to break the tie so the two halves start out different. EM does the rest: examples drift to whichever half suits them, and within a few rounds they've separated cleanly.

And if a split was pointless — the slot really was one situation — EM simply re-merges them, and you've lost nothing. **Bad splits self-correct; good splits stick.** That's why you can afford to be crude.

WHAT THIS IS WORTH

On the full-scale experiment, plain EM recovered 76.5% of the menus. Split-merge took it to 99.7% — essentially perfect. For comparison, running plain EM twenty times from different random starts and keeping the best only reached 82.5%.

Keeping what works and disturbing only what's broken beats re-rolling the dice, by a wide margin.

14. Glossary

bit

One on/off switch. A 1 or a 0. Our inputs and outputs are lists of them.

sparse

Most bits are off. Only a handful are on at a time.

slot (also: state, component, a value of z)

One tally sheet. Stores one situation-and-output pair: counters for how many examples landed here and how many had each bit on.

hidden variable (z , latent variable)

Which slot an example came from. Called hidden because it isn't in your data — the machine invents it because it explains the pattern.

tally / count

A counter that goes up. Learning here is updating these. During EM they go up by fractions rather than whole numbers.

smoothing / pseudo-counts

Starting every counter at 1 instead of 0, so nothing is ever recorded as flatly impossible.

prior

A belief you hold before seeing data. In practice, a small adjustment to your counts or an extra term in your objective. The smoothing above is one.

posterior

Your belief after seeing the evidence. The "50% / 50% / 0%" from section 7 is a posterior over slots.

Bayes' rule

The formal version of: see evidence, update your belief about the hidden cause. It's the scoring-and-scaling of section 7.

Bayesian network

A diagram of what depends on what, plus a table of numbers per item. Ours is z pointing at every bit.

latent class model

The specific name for our shape: one hidden variable causing a pile of independent bits.

E step

Using the current tally sheets, guess which slot each example belongs to (as fractions).

M step

Using those fractional guesses, re-do the tallies.

EM

Alternating E and M until nothing changes. Counting, in a loop, for when the labels are missing.

local optimum

A settling point that can't be improved by small steps but isn't the best one. The small hill in the fog.

split-merge

Deliberately rearranging a settled model — splitting overloaded slots, fusing redundant ones — then re-running EM, to escape a local optimum.

multimodal

One input has several distinct valid outputs. Stored as several slots agreeing on the input and disagreeing on the output.

Everything above is implemented in `latent_class.py` and `splitmerge.py` in this folder. The tiny 8-example dataset here is a hand-sized version of the 20,000-example synthetic world those scripts build.